

AI Lab Practicals

Complete Study Guide with Viva Q&A

Practicals 1–5 | Simple Language | Exam Ready

Contents

Practical 1 DFS & BFS — Depth First Search / Breadth First Search

Practical 2 A* Algorithm — Shortest Path in a Grid

Practical 3 Prim's MST — Minimum Spanning Tree (Greedy)

Practical 4 N-Queens — Backtracking / Branch & Bound

Practical 5 Customer Chatbot — Rule-Based NLP

Practical 1 — DFS & BFS

Problem Statement: Implement Depth First Search (DFS) and Breadth First Search (BFS) algorithms using an undirected graph. Write both recursive and non-recursive versions to search all vertices of the graph.

What is the Graph?

A graph is a collection of nodes (vertices) connected by edges. An undirected graph means connections go both ways — if node A connects to B, then B also connects to A.

```
graph = {  
    0: [1, 2],  
    1: [0, 3, 4],  
    2: [0, 5],  
    3: [1],  
    4: [1, 5],  
    5: [2, 4]  
}
```

DFS — Depth First Search (Like exploring a maze)

DFS goes as DEEP as possible first. Start at node 0, keep going forward until you hit a dead end, then backtrack and try another path. Uses a STACK (or recursion).

RECURSIVE DFS:

```
def dfs_recursive(graph, node, visited=None):  
  
    if visited is None:  
        visited = set()          # create empty set first time  
  
    print(node, end=' ')         # print current node  
    visited.add(node)            # mark as visited  
  
    for neighbor in graph[node]:  
        if neighbor not in visited:    # only visit unvisited  
            dfs_recursive(graph, neighbor, visited) # go deeper!
```

Output: 0 1 3 4 5 2

The function calls ITSELF (recursion). Each call goes one level deeper. When no unvisited neighbours remain, the function returns — this is automatic backtracking.

ITERATIVE DFS (USES STACK):

```
def dfs_iterative(graph, start):  
  
    visited = set()  
  
    stack = [start]          # Stack: last in, first out  
  
    while stack:  
  
        node = stack.pop()    # pick TOP item  
  
        if node not in visited:  
  
            print(node, end=' ')  
  
            visited.add(node)  
  
            for neighbor in reversed(graph[node]): # reversed for order  
  
                stack.append(neighbor)
```

Output: 0 1 3 4 5 2

BFS — Breadth First Search (Level by level)

BFS visits all nodes at distance 1 first, then distance 2, etc. Uses a QUEUE (first in, first out). Like spreading water — covers nearest areas before going far.

```
def bfs_iterative(graph, start):  
  
    visited = set()  
  
    queue = deque([start])    # Queue: first in, first out  
  
    visited.add(start)  
  
    while queue:  
  
        node = queue.popleft() # pick FRONT item  
  
        print(node, end=' ')  
  
        for neighbor in graph[node]:  
  
            if neighbor not in visited:  
  
                visited.add(neighbor)  
  
                queue.append(neighbor) # add to BACK
```

Output: 0 1 2 3 4 5 (level by level!)

Feature	DFS	BFS
Data Structure	Stack	Queue
Approach	Go deep first	Go wide first (level by level)
Output order	0 1 3 4 5 2	0 1 2 3 4 5
Use case	Maze solving, cycle detection	Shortest path, social networks
Time complexity	$O(V + E)$	$O(V + E)$

Viva Questions & Answers

Q1. What is the difference between DFS and BFS?

DFS goes as deep as possible first using a Stack. BFS visits all nearby nodes first using a Queue. DFS output: 0 1 3 4 5 2. BFS output: 0 1 2 3 4 5.

Q2. What data structure does DFS use? And BFS?

DFS uses a Stack (LIFO — Last In First Out). BFS uses a Queue (FIFO — First In First Out). In recursive DFS, the program's own call stack acts as the stack.

Q3. What is a visited set and why do we need it?

The visited set tracks nodes already seen. Without it, we'd keep visiting the same nodes forever in an infinite loop because the graph has cycles.

Q4. What is an undirected graph?

A graph where edges have no direction. If A connects to B, then B also connects to A. In code: `graph[u].append(v)` AND `graph[v].append(u)`.

Q5. What is backtracking in DFS?

When DFS reaches a node with no unvisited neighbours, it returns to the previous node and tries other neighbours. In recursive DFS, this happens automatically when the function returns.

Q6. What is the time complexity of DFS and BFS?

Both are $O(V + E)$ — V = vertices, E = edges. We visit each node once and check each edge once.

Q7. When would you use DFS over BFS in real life?

DFS: detecting cycles, topological sorting, maze solving. BFS: shortest path, level-order traversal, finding friends-of-friends in social networks.

Practical 2 — A* Algorithm

Problem Statement: Implement the A* (A-star) algorithm for a game search problem — specifically, finding the shortest path in a grid from a start point to a goal point, avoiding walls.

Core Idea — How A* Works

A* is like a smart GPS. Instead of blindly exploring all directions (like BFS), it uses a 'guess' about how far the goal is to always explore the most promising path first.

```
f(n) = g(n) + h(n)
```

```
g(n) = actual cost from start to current node (steps taken so far)
```

```
h(n) = heuristic: estimated cost from current node to goal (a guess)
```

```
f(n) = total estimated cost. Node with LOWEST f is explored next.
```

Heuristic — Manhattan Distance

```
def h(a, b):
```

```
    return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

```
# Example: from (0,0) to (2,2):
```

```
# |0-2| + |0-2| = 2 + 2 = 4
```

```
# Works when we can only move in 4 directions (no diagonal)
```

Manhattan distance = how many steps in rows + how many steps in columns. Named after Manhattan's grid-like streets.

The Grid

```
grid = [
```

```
    [0, 0, 0],    # row 0: all walkable
```

```
    [1, 0, 1],    # row 1: walls at col 0 and col 2 (blocked!)
```

```
    [0, 0, 0],    # row 2: all walkable
```

```
]
```

```
start = (0, 0)    goal = (2, 2)
```

Algorithm Steps

Step	Description
Step 1	Put start node (0,0) in a priority queue (min-heap) with f=0
Step 2	Pop node with lowest f. Explore its 4 neighbours (up/down/left/right only)

Step 3	For each valid neighbour: calculate g (steps so far +1) and h (Manhattan to goal). Store parent.
Step 4	If we reach the goal, trace back through parents to get the full path
Step 5	Output: [(0,0), (0,1), (1,1), (2,1), (2,2)]

Path explanation: Start at (0,0) → go right to (0,1) → down to (1,1) — the ONLY open cell in row 1 (walls block (1,0) and (1,2)) → down to (2,1) → right to goal (2,2).

Viva Questions & Answers

Q1. What does A* stand for and what makes it special?

A* (A-star) is a best-first search algorithm. It uses a heuristic to guide the search toward the goal, making it much faster than BFS on large grids while still finding the optimal (shortest) path.

Q2. What is a heuristic? What heuristic is used here?

A heuristic is an estimated cost from current position to goal. Here we use Manhattan distance: $|\text{row1} - \text{row2}| + |\text{col1} - \text{col2}|$. It works when movement is restricted to 4 directions (no diagonal).

Q3. What is $f(n) = g(n) + h(n)$?

f = total estimated cost. g = actual cost from start to current node. h = estimated cost to goal. A* always picks the node with the smallest f value next.

Q4. What data structure is used and why?

A min-heap (priority queue using `heapq`). It always gives us the node with the lowest f value in $O(\log n)$ time, which is what A* needs to explore the best node first.

Q5. What is an admissible heuristic?

A heuristic that never overestimates the actual cost. Manhattan distance is admissible — the real path length is always $\geq$ Manhattan distance. This guarantees A* finds the optimal path.

Q6. What does the parent dictionary do?

It stores which node we came from to reach each node. Once we reach the goal, we trace backwards through parents to reconstruct the full path from start to goal.

Q7. Difference between A* and Dijkstra's algorithm?

Dijkstra explores all directions equally ($h=0$, no heuristic). A* adds a heuristic to prioritize nodes closer to the goal. A* is faster with a good heuristic; Dijkstra is used when no heuristic is available.

Practical 3 — Prim's MST (Greedy Algorithm)

Problem Statement: Implement the Greedy search algorithm for Prim's Minimal Spanning Tree (MST). Also implement Kruskal's algorithm for comparison.

What is a Minimum Spanning Tree?

Imagine connecting 5 cities with roads. You want ALL cities connected but with MINIMUM total road length (cost). The MST gives you exactly that — connect all nodes, no cycles, minimum total weight.

Key fact: An MST for n vertices always has exactly $n-1$ edges.

Prim's Algorithm — How It Works

Step 1	Start from any vertex (vertex 0). Add it to the 'visited' set.
Step 2	Look at all edges going OUT from visited nodes. Add them to a min-heap (sorted by weight).
Step 3	Pick the CHEAPEST edge whose destination is NOT yet visited. Add that destination to visited.
Step 4	Add this edge to the MST. Repeat from Step 2 until all vertices are visited.

KEY CODE — PRIM'S:

```
import heapq    # min-heap: always gives smallest item first

visited = set([start])    # already-visited nodes

min_heap = []            # priority queue of (weight, from, to)

# Push all edges from start

heapq.heappush(min_heap, (weight, start, neighbor))

# Main loop

weight, u, v = heapq.heappop(min_heap) # get cheapest edge

if v not in visited:        # destination not visited yet?

    visited.add(v)

    edges.append((u, v, weight)) # this edge is in the MST!
```

RESULT FOR THE GIVEN INPUT:

Input: 5 vertices, 6 edges

Edges: 0-1(2), 0-3(6), 1-2(3), 1-3(8), 1-4(5), 2-4(7)

Prim's MST (from vertex 0):

```
0 - 1 : 2    <- cheapest edge from vertex 0

1 - 2 : 3    <- next cheapest connecting new node

1 - 4 : 5    <- next

0 - 3 : 6    <- connects last node
```

Total MST cost = 2 + 3 + 5 + 6 = 16

Kruskal's Algorithm (Bonus):

Kruskal's sorts ALL edges by weight first, then adds cheapest ones that don't form a cycle. Uses Union-Find to detect cycles. Both algorithms give the SAME MST!

```
# Sort all edges by weight

edges.sort(key=lambda x: x[2])

# Add edge only if it doesn't form a cycle

if find(parent, u) != find(parent, v): # different components?

    union(parent, rank, u, v)          # merge them

    mst.append((u, v, weight))         # add to MST

# Kruskal's output same: 0-1:2, 1-2:3, 1-4:5, 0-3:6
```

Viva Questions & Answers

Q1. What is a Minimum Spanning Tree (MST)?

An MST connects all vertices of a weighted graph with the minimum possible total edge weight, without forming any cycle. For n vertices, it always has exactly $n-1$ edges.

Q2. How does Prim's algorithm work?

Start from one vertex. Always pick the cheapest edge connecting a new (unvisited) vertex to the already-visited group. Repeat until all vertices are included. We use a min-heap for efficiency.

Q3. What data structure does Prim's use and why?

A min-heap (priority queue). It gives us the minimum weight edge in $O(\log n)$ time, instead of scanning all edges every time which would be $O(n)$.

Q4. Difference between Prim's and Kruskal's algorithm?

Prim's grows from one vertex, always picking cheapest edge connected to the current tree. Kruskal's sorts all edges globally and picks cheapest ones that don't form a cycle. Both give the same MST.

Q5. What is Union-Find used for in Kruskal's?

Union-Find detects if adding an edge would create a cycle. If both endpoints already belong to the same connected component (same 'set'), adding that edge would form a cycle — so we skip it.

Q6. What is a greedy algorithm? Is Prim's greedy?

A greedy algorithm makes the best local choice at each step. Yes, Prim's is greedy — it always picks the minimum weight edge available. Interestingly, it still achieves the globally optimal MST.

Q7. How many edges does an MST have for n vertices?

Always exactly $n-1$ edges. In our example: 5 vertices $\rightarrow$ 4 edges (2+3+5+6). This is because to connect n nodes without any cycle, you need exactly $n-1$ connections.

Practical 4 — N-Queens (Backtracking)

Problem Statement: Implement a solution for a Constraint Satisfaction Problem (CSP) using Branch and Bound and Backtracking — specifically the N-Queens problem where N queens must be placed on an N×N chessboard such that no two queens attack each other.

Understanding the Problem

A queen in chess can attack any piece in the same row, same column, or same diagonal. We need to place N queens so no queen can attack any other queen. For N=4: only 2 valid arrangements exist.

```
Solution 1:          Solution 2:

0 Q 0 0              0 0 Q 0
0 0 0 Q              Q 0 0 0
Q 0 0 0              0 0 0 Q
0 0 Q 0              0 Q 0 0

board = [1, 3, 0, 2]  board = [2, 0, 3, 1]

(queen in col 1,3,0,2) (queen in col 2,0,3,1)
```

The Safety Check — is_safe()

```
def is_safe(board, row, col, n):

    for i in range(row):          # check all rows ABOVE

        if board[i] == col:       # same column? not safe!

            return False

        # same diagonal? if row diff == col diff → diagonal

        if abs(board[i] - col) == abs(i - row):

            return False

    return True    # safe to place queen here!

# board[i] = column where queen is placed in row i

# We only check above because queens are placed row by row
```

Backtracking Logic — backtrack()

```
def backtrack(row):
```

```

    if row == n:          # all n queens placed! Save solution.

        solutions.append(board.copy())

        return

    for col in range(n):  # try every column in this row

        if is_safe(board, row, col, n):

            board[row] = col    # place queen at (row, col)

            backtrack(row + 1) # try to place next queen

            board[row] = -1     # BACKTRACK: undo placement!

# board[row] = -1 means 'no queen in this row' (reset/undo)

```

Backtracking = 'Try something. If it leads to failure, undo it and try something else.' Like solving a Sudoku puzzle — if a number doesn't work, erase it and try another.

What is a CSP?

CSP Component	In N-Queens
Variables	Each row (Row 0, Row 1, ... Row N-1)
Domain	Possible column positions: 0 to N-1
Constraints	No two queens in same column or diagonal
Solution	An assignment satisfying all constraints

Viva Questions & Answers

Q1. What is the N-Queens problem?

Place N queens on an N×N chessboard so no two queens attack each other (no same row, column, or diagonal). For N=4: 2 solutions. For N=8: 92 solutions.

Q2. What is backtracking?

Backtracking tries a placement, and if it leads to failure, undoes the last step and tries a different option. Like exploring a tree of possibilities but cutting off dead branches early to save time.

Q3. Why do we check only rows above in is_safe()?

Because we place queens row by row from top to bottom. Rows below have no queens yet, so there's nothing to check. This makes the check efficient — O(row) instead of O(n).

Q4. How do you check if two queens are on the same diagonal?

If $|row1 - row2| == |col1 - col2|$. Example: queens at (0,1) and (2,3): $|0 - 2| = 2$, $|1 - 3| = 2 \rightarrow$ same diagonal, not safe. This checks both the left and right diagonals simultaneously.

Q5. Difference between Backtracking and Branch & Bound?

Backtracking abandons a path when a constraint is violated. Branch & Bound additionally uses a cost function to prune paths that can't lead to an optimal solution — used for optimization problems.

Q6. What is a Constraint Satisfaction Problem (CSP)?

A problem with variables (rows), domains (column values 0 to N-1), and constraints (no two queens attack each other). We find an assignment of values to all variables that satisfies every constraint.

Q7. Why is `board[row] = -1` at the end of `backtrack()`?

This is the 'undo' step. After exploring all options from a column, we reset that row to -1 (no queen placed) so the parent function can try the next column without interference from our previous attempt.

Practical 5 — Customer Chatbot

Problem Statement: Develop an elementary chatbot for a customer interaction application. The chatbot should take customer name, email, and issue as input, generate an automatic response based on keywords, and save all records to a CSV file.

How the Chatbot Works — 6 Steps

Step 1 CSV Setup	Creates customers.csv with columns: Name, Email, Issue, Response. Only creates the file if it doesn't already exist.
Step 2 chatbot_response()	The brain of the chatbot. Converts input to lowercase, then checks for keywords like 'payment', 'login', 'refund', 'delivery', 'damaged', 'crash'.
Step 3 add_customer()	Gets name, email, issue from user input. Calls chatbot_response() to get reply. Saves all 4 fields to the CSV file.
Step 4 view_customers()	Opens CSV file and prints all rows — shows the complete history of all customer interactions.
Step 5 menu()	A while loop showing: 1. Add Customer, 2. View Customers, 3. Exit. Runs continuously until user chooses Exit.
Step 6 Run	Calls menu() to start the program. Everything runs from this single call.

THE KEYWORD MATCHING LOGIC:

```
def chatbot_response(user_input):  
  
    user_input = user_input.lower() # 'PAYMENT' → 'payment'  
  
    if 'payment' in user_input:  
  
        return 'Please check your bank details or try again later.'  
  
    elif 'login' in user_input:  
  
        return 'Try resetting your password or check your credentials.'  
  
    elif 'delivery' in user_input or 'not delivered' in user_input:  
  
        return 'Your order is on the way. Please wait 2-3 days.'  
  
    elif 'refund' in user_input:  
  
        return 'Your refund will be processed within 5-7 business days.'  
  
    elif 'damaged' in user_input:  
  
        return 'We are sorry! Please request a replacement.'  
  
    elif 'crash' in user_input:  
  
        return 'Please update the app or reinstall it.'
```

```

elif 'coupon' in user_input:

    return 'Check coupon validity or try another code.'

elif 'wrong item' in user_input:

    return 'Please request a return or replacement.'

elif 'account locked' in user_input:

    return 'Wait 30 minutes or reset your password.'

else:

    return 'Sorry, I did not understand. Please contact support.'
```

SAMPLE OUTPUT FROM THE CODE:

Customer	Issue Entered	Response
Rahul	payment failed	Check bank details...
Priya	login problem	Reset your password...
Karan	app crash	Update/reinstall app...
Neha	refund not received	5-7 business days...
Awantika	cannot detect account details	Fallback response triggered

Suggested Improvement:

Awantika's issue 'cannot detect account details' triggered the fallback because no keyword matched. Adding this line before the else block would fix it:

```

# Add this before the 'else' block:

elif 'account' in user_input:

    return 'Please verify your account details or contact support.'
```

Now 'cannot detect account details' would match 'account'

and give a proper, relevant response instead of the fallback.

Viva Questions & Answers

Q1. What type of chatbot is this?

This is a rule-based (keyword-matching) chatbot. It doesn't use AI or machine learning — it checks if specific words appear in the input and returns pre-written replies. Simple but effective for predictable customer issues.

Q2. Why do we convert input to lowercase using .lower()?

So that 'PAYMENT', 'Payment', and 'payment' all match the same keyword. Without .lower(), the bot would miss messages typed in uppercase or mixed case.

Q3. What is a CSV file and why use it here?

CSV (Comma-Separated Values) is a simple text file storing data in rows with values separated by commas. We use it as a simple database to persistently store customer records — openable in Excel, needs no database software.

Q4. What does the 'a' mode in open() do? What about 'w'?

'a' = append mode: adds new rows to the END of the file without deleting existing data. 'w' = write mode: creates a fresh file (deletes everything). We use 'w' only for creating the initial file with just the header row.

Q5. What is the difference between rule-based and AI-based chatbots?

Rule-based chatbots use if/elif logic with fixed keywords — fast and predictable but can't handle new phrases. AI-based chatbots (like ChatGPT) use machine learning to understand natural language and varied phrasing.

Q6. What happens if the user types an issue with no matching keyword?

The else branch returns: 'Sorry, I didn't understand. Please contact support.' This is a fallback response. Awantika's issue 'cannot detect account details' triggered this because no keyword (payment, login, etc.) was found.

Q7. How can this chatbot be improved?

Add more keyword synonyms, use NLP libraries like NLTK for better understanding, use a real database instead of CSV, add sentiment analysis to detect frustrated customers, and implement ML-based intent classification.